

Smart Pharmacy Inventory and Prescription Management System

Authors: Mert Eldemir, Tuba Aydın

Course: Database Systems – COME312

1. Project Scenario

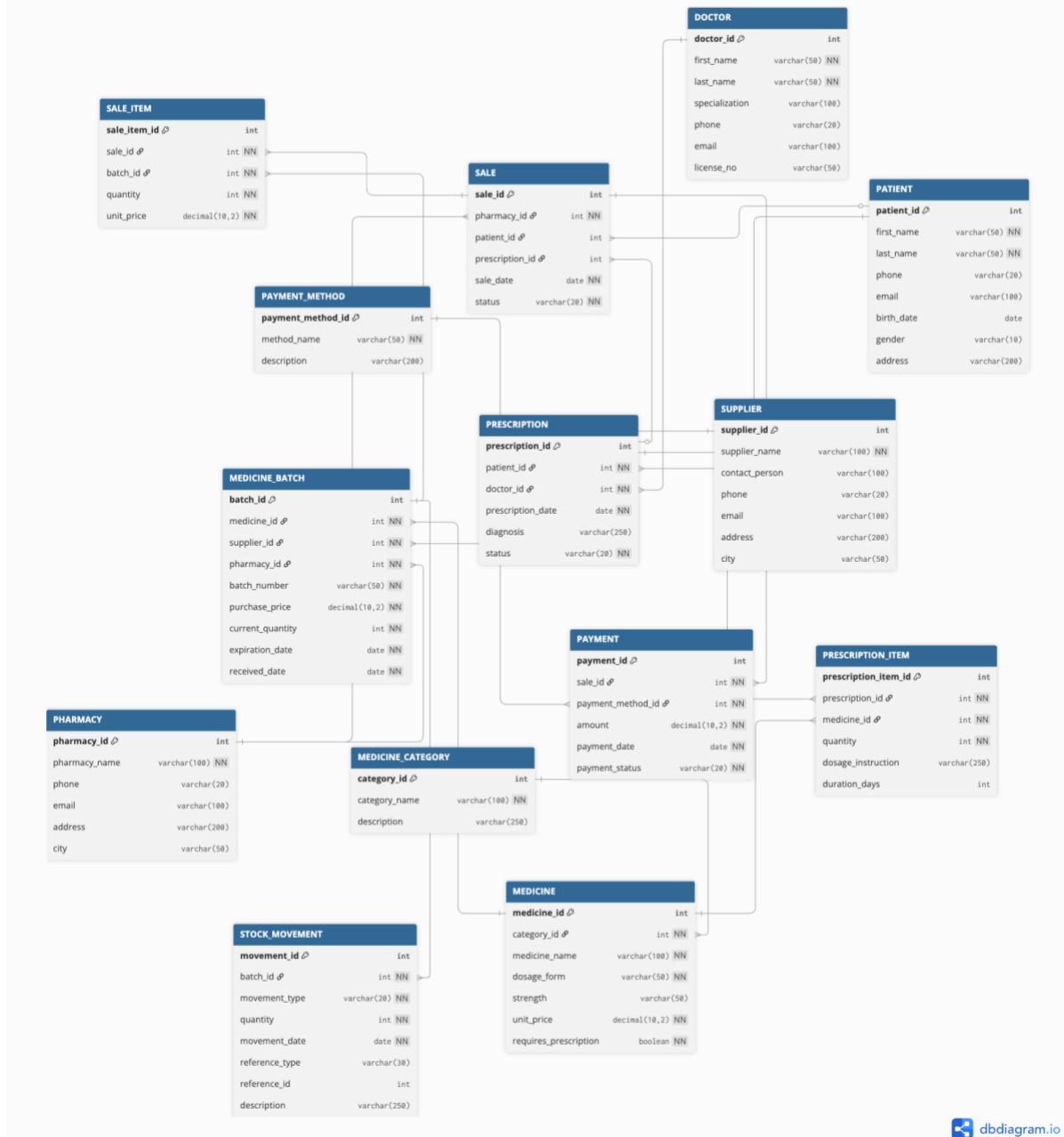


The **Smart Pharmacy Inventory and Prescription Management System** is designed to manage pharmacy operations such as **medicine inventory**, **supplier batches**, **prescriptions**, **medicine sales**, **payments**, and **stock movements**. The system stores **patient** and **doctor information**, tracks **medicines by category** and **batch**, records **prescription details**, processes **medicine sales**, and automatically **updates inventory levels**. It also supports reporting operations such as **low-stock medicines**, **expired batches**, **sales by date range**, **prescription history by patient**, and **supplier-based medicine stock reports**.

2. ER Diagram

2.1 Main Entities

1. PATIENT
2. DOCTOR
3. PHARMACY
4. SUPPLIER
5. MEDICINE_CATEGORY
6. MEDICINE
7. MEDICINE_BATCH
8. PRESCRIPTION
9. PRESCRIPTION_ITEM
10. SALE
11. SALE_ITEM
12. PAYMENT_METHOD
13. PAYMENT
14. STOCK_MOVEMENT



2.2 Core Relationships

- One patient can have many prescriptions.
- One doctor can write many prescriptions.
- One prescription can include many medicines.
- One medicine can appear in many prescriptions.
- One pharmacy can make many sales.
- One sale can include many medicines.
- One sale has one payment.
- One payment uses one payment method.
- One supplier can supply many medicine batches.
- One medicine can have many batches.
- One pharmacy can have many medicine batches.
- One medicine category can include many medicines.
- One medicine batch can have many stock movements.

3. Trigger Idea

When a new **SALE_ITEM** is inserted, the system automatically decreases the quantity of the related **MEDICINE_BATCH** and inserts a new record into **STOCK_MOVEMENT**.

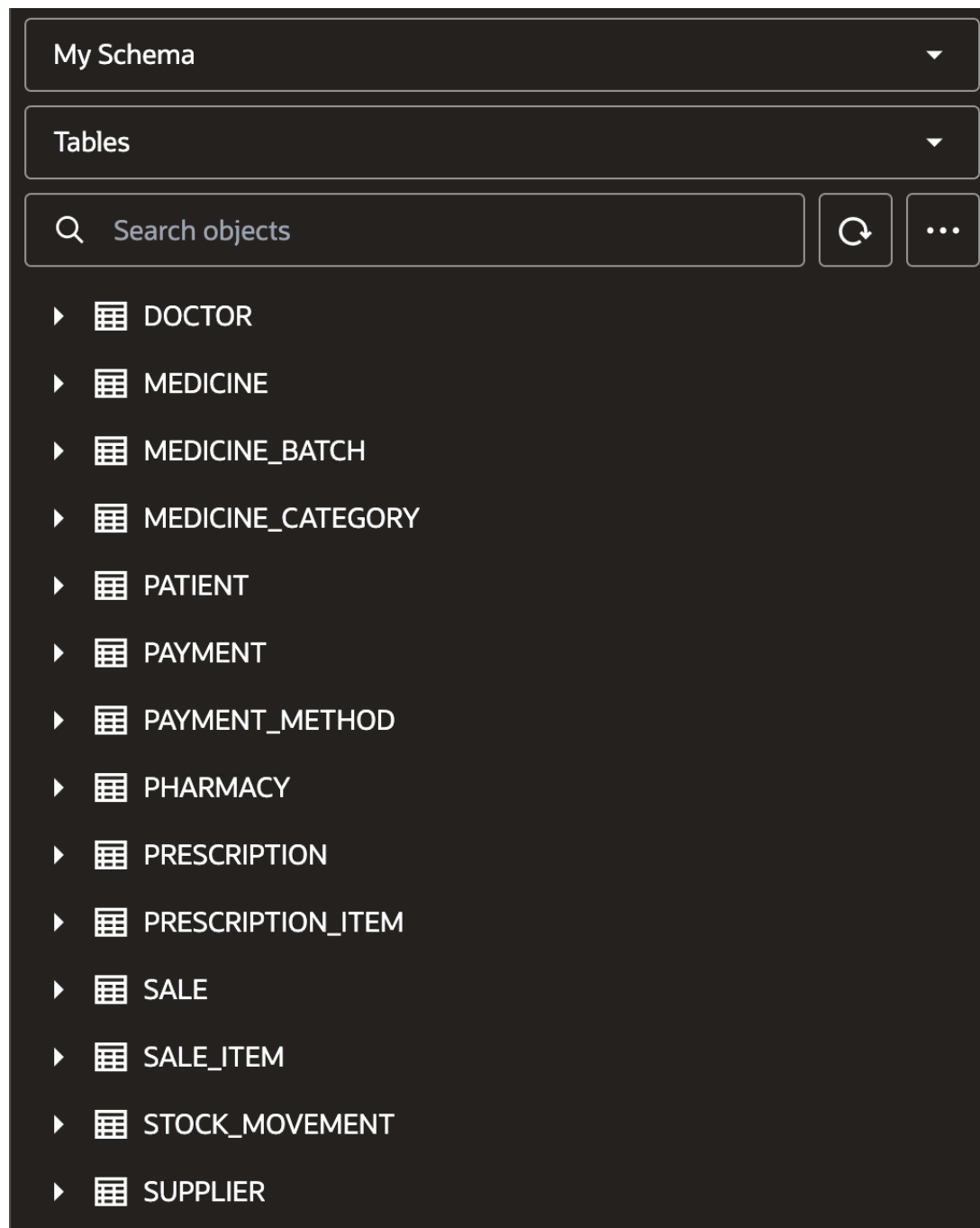
Example logic:

SALE_ITEM inserted:
Medicine Batch ID = 5
Quantity Sold = 2

Trigger does:

MEDICINE_BATCH.current_quantity decreases by 2
STOCK_MOVEMENT gets a new **OUT movement**

4. Oracle Create Tables



Example code for Patient, Doctor and Pharmacy table creation:

Database-Systems-COME312/blob/main/Project/Solution/Tables/Create-Tables.sql

```
1  --- Patient
2  CREATE TABLE PATIENT (
3      PATIENT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
4      FIRST_NAME VARCHAR2(50) NOT NULL,
5      LAST_NAME  VARCHAR2(50) NOT NULL,
6      PHONE     VARCHAR2(20),
7      EMAIL     VARCHAR2(100),
8      BIRTH_DATE DATE,
9      GENDER    VARCHAR2(10),
10     ADDRESS   VARCHAR2(200),
11     CONSTRAINT UK_PATIENT_EMAIL UNIQUE ( EMAIL ),
12     CONSTRAINT CHK_PATIENT_GENDER
13     | CHECK ( GENDER IN ( 'Male', 'Female', 'Other' ) )
14 );
15
16 --- Doctor
17 CREATE TABLE DOCTOR (
18     DOCTOR_ID    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
19     FIRST_NAME   VARCHAR2(50) NOT NULL,
20     LAST_NAME    VARCHAR2(50) NOT NULL,
21     SPECIALIZATION VARCHAR2(100),
22     PHONE        VARCHAR2(20),
23     EMAIL        VARCHAR2(100),
24     LICENSE_NO   VARCHAR2(50),
25     CONSTRAINT UK_DOCTOR_EMAIL UNIQUE ( EMAIL ),
26     CONSTRAINT UK_DOCTOR_LICENSE UNIQUE ( LICENSE_NO )
27 );
28
29 --- Pharmacy
30 CREATE TABLE PHARMACY (
31     PHARMACY_ID    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
32     PHARMACY_NAME  VARCHAR2(100) NOT NULL,
33     PHONE          VARCHAR2(20),
34     EMAIL          VARCHAR2(100),
35     ADDRESS        VARCHAR2(200),
36     CITY           VARCHAR2(50),
37     CONSTRAINT UK_PHARMACY_EMAIL UNIQUE ( EMAIL )
38 );
39
```

4.2 Create Indexes

Even though **primary keys**, **foreign keys**, **unique constraints**, and **check constraints** are already included as **indexes in Database**, for the study purpose we can define indexes by ourselves:

Database-Systems-COME312/blob/main/Project/Solution/Tables/Create-Indexes.sql

```
1  -- Medicine and inventory indexes --
2
3  CREATE INDEX IDX_MEDICINE_CATEGORY ON
4      MEDICINE (
5          CATEGORY_ID
6      ); -- for joining MEDICINE with MEDICINE_CATEGORY
7  CREATE INDEX IDX_BATCH_MEDICINE ON
8      MEDICINE_BATCH (
9          MEDICINE_ID
10     ); -- for finding batches of a specific medicine
11  CREATE INDEX IDX_BATCH_SUPPLIER ON
12      MEDICINE_BATCH (
13          SUPPLIER_ID
14     ); -- for supplier-based inventory reports.
15  CREATE INDEX IDX_BATCH_PHARMACY ON
16      MEDICINE_BATCH (
17          PHARMACY_ID
18     ); -- for pharmacy-based stock reports
19  CREATE INDEX IDX_BATCH_EXPIRATION_DATE ON
20      MEDICINE_BATCH (
21          EXPIRATION_DATE
22     ); -- for expired / soon-expiring medicine reports
23
```

5. Create PL/SQL package specification

This is the “**interface**” of the **package**.

It declares **procedure/function names**:

<Database-Systems-COME312/blob/main/Project/Solution/Package/Package-Spec.sql>

```
1  -- PACKAGE SPECIFICATION
2  CREATE
3  OR REPLACE PACKAGE PKG_PHARMACY_SYSTEM AS
4  -- Patient operations
5  PROCEDURE ADD_PATIENT (
6      P_FIRST_NAME IN PATIENT.FIRST_NAME,
7      P_LAST_NAME IN PATIENT.LAST_NAME,
8      P_PHONE IN PATIENT.PHONE,
9      P_EMAIL IN PATIENT.EMAIL,
10     P_BIRTH_DATE IN PATIENT.BIRTH_DATE,
11     P_GENDER IN PATIENT.GENDER,
12     P_ADDRESS IN PATIENT.ADDRESS
13 );
14
15 PROCEDURE UPDATE_PATIENT (
16     P_PATIENT_ID IN PATIENT.PATIENT_ID,
17     P_FIRST_NAME IN PATIENT.FIRST_NAME,
18     P_LAST_NAME IN PATIENT.LAST_NAME,
19     P_PHONE IN PATIENT.PHONE,
20     P_EMAIL IN PATIENT.EMAIL,
21     P_BIRTH_DATE IN PATIENT.BIRTH_DATE,
22     P_GENDER IN PATIENT.GENDER,
23     P_ADDRESS IN PATIENT.ADDRESS
24 );
25
26 PROCEDURE DELETE_PATIENT (P_PATIENT_ID IN PATIENT.PATIENT_ID);
27
28 -- Medicine operations
29 PROCEDURE ADD_MEDICINE (
30     P_CATEGORY_ID IN MEDICINE.CATEGORY_ID,
31     P_MEDICINE_NAME IN MEDICINE.MEDICINE_NAME,
32     P_DOSAGE_FORM IN MEDICINE.DOSAGE_FORM,
33     P_STRENGTH IN MEDICINE.STRENGTH,
34     P_UNIT_PRICE IN MEDICINE.UNIT_PRICE,
35     P_REQUIRES_PRESCRIPTION IN MEDICINE.REQUIRES_PRESCRIPTION
36 );
37
38 PROCEDURE UPDATE_MEDICINE (
39     P_MEDICINE_ID IN MEDICINE.MEDICINE_ID,
40     P_CATEGORY_ID IN MEDICINE.CATEGORY_ID,
41     P_MEDICINE_NAME IN MEDICINE.MEDICINE_NAME,
42     P_DOSAGE_FORM IN MEDICINE.DOSAGE_FORM,
43     P_STRENGTH IN MEDICINE.STRENGTH,
44     P_UNIT_PRICE IN MEDICINE.UNIT_PRICE,
45     P_REQUIRES_PRESCRIPTION IN MEDICINE.REQUIRES_PRESCRIPTION
46 );
47
48 PROCEDURE DELETE_MEDICINE (P_MEDICINE_ID IN MEDICINE.MEDICINE_ID);
49
```

Create **PL/SQL package body**:

This is where we write the **actual logic**

<Database-Systems-COME312/blob/main/Project/Solution/Package/Package-Body.sql>

```
1  -- PACKAGE BODY
2
3  CREATE OR REPLACE PACKAGE BODY PKG_PHARMACY_SYSTEM AS
4  -- Patient operations
5  PROCEDURE ADD_PATIENT (
6      P_FIRST_NAME IN PATIENT.FIRST_NAME%TYPE,
7      P_LAST_NAME IN PATIENT.LAST_NAME%TYPE,
8      P_PHONE IN PATIENT.PHONE%TYPE,
9      P_EMAIL IN PATIENT.EMAIL%TYPE,
10     P_BIRTH_DATE IN PATIENT.BIRTH_DATE%TYPE,
11     P_GENDER IN PATIENT.GENDER%TYPE,
12     P_ADDRESS IN PATIENT.ADDRESS%TYPE
13 ) AS
14 BEGIN
15     INSERT INTO PATIENT (
16         FIRST_NAME,
17         LAST_NAME,
18         PHONE,
19         EMAIL,
20         BIRTH_DATE,
21         GENDER,
22         ADDRESS
23     ) VALUES ( P_FIRST_NAME,
24                 P_LAST_NAME,
25                 P_PHONE,
26                 P_EMAIL,
27                 P_BIRTH_DATE,
28                 P_GENDER,
29                 P_ADDRESS );
30
31     DBMS_OUTPUT.PUT_LINE('Patient inserted successfully.');
```

6. Create Trigger

Our trigger should be:

When a **SALE_ITEM** is inserted:

1. Check whether **enough stock exists**.
2. Decrease **MEDICINE_BATCH.current_quantity**.
3. Insert an **OUT** record into **STOCK_MOVEMENT**.

<Database-Systems-COME312/blob/main/Project/Solution/Triggers/Create-Trigger.sql>

```
9 CREATE OR REPLACE TRIGGER TRG_SALE_ITEM_STOCK_MOVEMENT BEFORE
10 INSERT ON SALE_ITEM
11 FOR EACH ROW
12 DECLARE
13     V_CURRENT_QUANTITY MEDICINE_BATCH.CURRENT_QUANTITY%TYPE;
14 BEGIN
15     -- Get the current stock quantity of the selected batch.
16     SELECT
17         CURRENT_QUANTITY
18     INTO V_CURRENT_QUANTITY
19     FROM
20         MEDICINE_BATCH
21     WHERE
22         BATCH_ID = :NEW.BATCH_ID;
23
24     -- Check whether there is enough stock.
25     IF V_CURRENT_QUANTITY < :NEW.QUANTITY THEN
26         RAISE_APPLICATION_ERROR(-20001, 'Not enough stock available for this medicine batch.');
```

7. Create Test

1. This script inserts **sample** data and **tests**:
2. Package **insert procedures**
3. Trigger for **sale_item** -> **stock_movement**
4. **Dynamic report** procedures
5. Duplicate patient cleanup procedure

<Database-Systems-COME312/blob/main/Project/Solution/Test/Test-Data.sql>

8. Test Package Operations

It will include **update**, **delete**, **duplicate cleanup**, **insufficient stock test**, and **final report** calls.

<Database-Systems-COME312/blob/main/Project/Solution/Test/Test-Package-Operations.sql>

8.1 Update operation through package

We need to show that the package can update records.

```
15 DECLARE
16     V_PATIENT_ID PATIENT.PATIENT_ID%TYPE;
17 BEGIN
18     SELECT
19         PATIENT_ID
20     INTO V_PATIENT_ID
21     FROM
22         PATIENT
23     WHERE
24         EMAIL = 'ali.kaya@example.com';
25
26     PKG_PHARMACY_SYSTEM.UPDATE_PATIENT(
27         P_PATIENT_ID => V_PATIENT_ID,
28         P_FIRST_NAME => 'Ali',
29         P_LAST_NAME  => 'Kaya',
30         P_PHONE      => '05551112233',
31         P_EMAIL      => 'ali.kaya@example.com',
32         P_BIRTH_DATE => DATE '1998-05-12',
33         P_GENDER     => 'Male',
34         P_ADDRESS    => 'Updated Istanbul Address'
35     );
36
37 END;
38 /
```

Result

Patient updated successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.013



Check Updated Patient:

```
41 SELECT
42     PATIENT_ID,
43     FIRST_NAME,
44     LAST_NAME,
45     PHONE,
46     EMAIL,
47     ADDRESS
48 FROM
49     PATIENT
50 WHERE
51     EMAIL = 'ali.kaya@example.com';
52
```

	PATIENT_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL	ADDRESS
1	1	Ali	Kaya	05551112233	ali.kaya@example.co	Updated Istanbul Adr

8.2 Delete operation through package

We should test **delete on a safe record, not a record used by prescriptions/sales.**

For example, we inserted duplicate **Merve Sahin** patients. After testing **duplicate cleanup**, we can delete **a test patient** or create a **temporary patient** and delete it.

```

54 -- First insert a temporary patient, then delete it using package.
55 BEGIN
56     PKG_PHARMACY_SYSTEM.ADD_PATIENT(
57         P_FIRST_NAME => 'Temporary',
58         P_LAST_NAME  => 'Patient',
59         P_PHONE       => '05558889999',
60         P_EMAIL       => 'temporary.patient@example.com',
61         P_BIRTH_DATE  => DATE '1999-01-01',
62         P_GENDER      => 'Other',
63         P_ADDRESS     => 'Temporary Address'
64     );
65 END;
66 /
67

```

Delete Temporary inserted Patient:

```

69 SELECT
70     PATIENT_ID,
71     FIRST_NAME,
72     LAST_NAME,
73     EMAIL
74 FROM
75     PATIENT
76 WHERE
77     EMAIL = 'temporary.patient@example.com';
78
79 DECLARE
80     V_PATIENT_ID PATIENT.PATIENT_ID%TYPE;
81 BEGIN
82     SELECT
83         PATIENT_ID
84     INTO V_PATIENT_ID
85     FROM
86         PATIENT
87     WHERE
88         EMAIL = 'temporary.patient@example.com';
89
90     PKG_PHARMACY_SYSTEM.DELETE_PATIENT(P_PATIENT_ID => V_PATIENT_ID);
91 END;
92 /
93

```

8.3 Duplicate cleanup procedure

Duplicate rule:
same **first_name** + **last_name** + **phone**

```

109 -- Before cleanup: Merve Sahin should appear as duplicate.
110 SELECT
111     FIRST_NAME,
112     LAST_NAME,
113     PHONE,
114     COUNT(*) AS RECORD_COUNT
115 FROM
116     PATIENT
117 GROUP BY
118     FIRST_NAME,
119     LAST_NAME,
120     PHONE
121 HAVING
122     COUNT(*) > 1
123 ORDER BY
124     FIRST_NAME,
125     LAST_NAME;
126
127 BEGIN
128     PKG_PHARMACY_SYSTEM.REMOVE_DUPLICATE_PATIENTS;
129 END;
130 /

```


Result

Result size: 1

	LAST_NAME	PHONE	RECORD_COUNT
1	Sahin	05559990000	2

After Cleanup:

```
132  -- After cleanup: duplicate groups should be removed.
133  SELECT
134      FIRST_NAME,
135      LAST_NAME,
136      PHONE,
137      COUNT(*) AS RECORD_COUNT
138  FROM
139      PATIENT
140  GROUP BY
141      FIRST_NAME,
142      LAST_NAME,
143      PHONE
144  HAVING
145      COUNT(*) > 1
146  ORDER BY
147      FIRST_NAME,
148      LAST_NAME;
149
```

Result

No data found.

8.4 Trigger error test: insufficient stock

Vitamin C batch has low stock. We try to sell 999 units.

Expected result:

ORA-20001: Not enough stock available for this medicine batch.

The error is caught so the script can continue.

```

155 DECLARE
156     V_SALE_ID     SALE.SALE_ID%TYPE;
157     V_BATCH_VITC  MEDICINE_BATCH.BATCH_ID%TYPE;
158 BEGIN
159     SELECT
160         SALE_ID
161     INTO V_SALE_ID
162     FROM
163         SALE
164     WHERE
165         ROWNUM = 1;
166
167     SELECT
168         BATCH_ID
169     INTO V_BATCH_VITC
170     FROM
171         MEDICINE_BATCH
172     WHERE
173         BATCH_NUMBER = 'BATCH-VITC-001';
174
175     PKG_PHARMACY_SYSTEM.ADD_SALE_ITEM(
176         P_SALE_ID     => V_SALE_ID,
177         P_BATCH_ID     => V_BATCH_VITC,
178         P_QUANTITY     => 999,
179         P_UNIT_PRICE   => 90.00
180     );
181
182 EXCEPTION
183     WHEN OTHERS THEN
184         DBMS_OUTPUT.PUT_LINE('Expected trigger error caught: ' || SQLERRM);
185 END;
186 /

```

Result:

Expected trigger error caught: ORA-20001: Not enough stock available for this medicine batch.
 ORA-06512: at "MERT_ELDEMIRR7_SCHEMA_W7JW5.TRG_SALE_ITEM_STOCK_MOVEMENT", line
 15
 ORA-04088: error during execution of trigger
 'MERT_ELDEMIRR7_SCHEMA_W7JW5.TRG_SALE_ITEM_STOCK_MOVEMENT'

8.5 Reports after sale exists

```

202 BEGIN
203     PKG_PHARMACY_SYSTEM.GET_SALES_REPORT(
204         P_START_DATE => DATE '2026-02-01',
205         P_END_DATE   => DATE '2026-02-28',
206         P_PHARMACY_ID => NULL,
207         P_STATUS      => 'COMPLETED'
208     );
209
210     PKG_PHARMACY_SYSTEM.GET_LOW_STOCK_REPORT(
211         P_PHARMACY_ID => NULL,
212         P_MIN_QUANTITY => 10
213     );
214     PKG_PHARMACY_SYSTEM.GET_PRESCRIPTION_REPORT(
215         P_PATIENT_ID => NULL,
216         P_DOCTOR_ID  => NULL,
217         P_STATUS     => 'ACTIVE'
218     );
219
220 END;
221 /
222

```

Result:

--- SALES REPORT ---
 Sale ID: 1 | Date: 2026-02-07 | Pharmacy: Central Pharmacy | Patient: Ali Kaya | Status: COMPLETED |
 Total: 250
 --- LOW STOCK REPORT ---

Batch ID: 3 | Medicine: Vitamin C 1000 mg | Pharmacy: Health Plus Pharmacy | Batch No: BATCH-VITC-001 | Quantity: 8 | Expiration: 2027-09-30

--- PRESCRIPTION REPORT ---

Prescription ID: 1 | Date: 2026-02-05 | Patient: Ali Kaya | Doctor: Mehmet Aydin | Status: ACTIVE |

Diagnosis: Bacterial throat infection

Prescription ID: 2 | Date: 2026-02-06 | Patient: Zeynep Demir | Doctor: Elif Yilmaz | Status: ACTIVE |

Diagnosis: Allergic rhinitis

8.6 Sale Total Calculated Dynamically

We do not store **SALE.total_amount** or **SALE_ITEM.line_total**.

Sale total is **calculated** as **SUM(quantity * unit_price)**.

```
227 SELECT
228     S.SALE_ID,
229     S.SALE_DATE,
230     PH.PHARMACY_NAME,
231     S.STATUS,
232     SUM(SI.QUANTITY * SI.UNIT_PRICE) AS CALCULATED_SALE_TOTAL
233 FROM
234     | SALE S
235     JOIN PHARMACY PH ON S.PHARMACY_ID = PH.PHARMACY_ID
236     JOIN SALE_ITEM SI ON S.SALE_ID = SI.SALE_ID
237 GROUP BY
238     S.SALE_ID,
239     S.SALE_DATE,
240     PH.PHARMACY_NAME,
241     S.STATUS
242 ORDER BY
243     S.SALE_ID;
244
```

Result

Result size: 1

		PHARMACY_NAME	STATUS	CALCULATED_SAL
1	00	Central Pharmacy	COMPLETED	250

Parol: $2 \times 35 = 70$

Augmentin: $1 \times 180 = 180$

Total = 250

8.7 Stock Movements Verification

Final verification: **stock movements**

There should be **IN movements** from batch receiving

and **OUT movements** created automatically by the trigger.

```

248 SELECT
249     MOVEMENT_ID,
250     BATCH_ID,
251     MOVEMENT_TYPE,
252     QUANTITY,
253     MOVEMENT_DATE,
254     REFERENCE_TYPE,
255     REFERENCE_ID,
256     DESCRIPTION
257 FROM
258     STOCK_MOVEMENT
259 ORDER BY
260     MOVEMENT_ID;
261

```

Result:

	MOVEMENT_ID	BATCH_ID	MOVEMENT_TYPE	QUANTITY	MOVEMENT_DATE	REFERENCE_TYPE	REFERENCE_ID	DESCRIPTION
1	1	1	IN	100	1/10/2026, 12:00:00 AM	BATCH_RECEIVE	1	Initial stock received from supplier.
2	2	2	IN	50	1/15/2026, 12:00:00 AM	BATCH_RECEIVE	2	Initial stock received from supplier.
3	3	3	IN	8	1/20/2026, 12:00:00 AM	BATCH_RECEIVE	3	Initial stock received from supplier.
4	4	4	IN	30	1/25/2026, 12:00:00 AM	BATCH_RECEIVE	4	Initial stock received from supplier.
5	21	1	OUT	2	5/4/2026, 2:47:02 PM	SALE_ITEM	1	Stock decreased automatically after
6	22	2	OUT	1	5/4/2026, 2:47:02 PM	SALE_ITEM	1	Stock decreased automatically after

9. Final 3NF Conclusion

The **Smart Pharmacy Inventory and Prescription Management System** satisfies **Third Normal Form** because:

1. Each table has a **primary key**.
2. Each column **contains atomic values**.
3. **Repeating groups** are **separated** into **detail tables**.
4. **Non-key attributes** depend on the **primary key**.
5. There are **no partial dependencies**.
6. There are **no transitive dependencies**.
7. **Lookup/definition** data is **stored in separate tables**.
8. **Calculated** values such as **sale total** and **line total** are **not stored**.

Therefore, the database design is normalized up to **Third Normal Form (3NF)** and is suitable for reliable pharmacy inventory, prescription, sales, payment, and stock movement management.

Source Code of the Project: [Database-Systems-COME312/Project](#)